

Reserve identifiers preceded by @ for non-ignorable annotation tokens

Document #: P3254R0
Date: 2024-05-13
Project: Programming Language C++
Audience: EWG
Reply-to: Brian Bi
<bbi10@bloomberg.net>

Contents

- 1 Abstract
- 2 Introduction
- 3 Prior art in other programming languages
- 4 Example: contract annotations
- 5 Example: trivial relocatability
- 6 Example: profile annotations
- 7 Example: `[[no_unique_address]]`
- 8 Making `@identifier` a preprocessing token
- 9 Scoped annotation keys
- 10 Wording
- 11 Policy for use of *annotation-keys*
- 12 Acknowledgements
- 13 References

§ 1 Abstract

[[P2558R2](#)], adopted into C++23, added the @ character to the basic source character set. There are no currently valid tokens containing this character, other than literals. I argue that using @ to introduce a non-ignorable annotation is a plausible future use of the character and give some plausible examples (but do not formally propose any). In order to leave this syntax space open, I propose that at present, @ directly followed by an identifier be a single preprocessing token.

§ 2 Introduction

Attributes were originally envisioned as “a way to open up a new namespace for keywords reserved to the implementers” and avoid “risk of collision with existing users’ names” [N2224]. The proposal for attribute syntax that was ultimately adopted into the Standard, [N2761], suggested that attributes be used only for “minor annotations” (and, thus, not completely eliminate the need to introduce new keywords, or new meanings for existing keywords, to designate new major features).

Both papers’ objectives have been frustrated to a great extent by the eventual emergence of the attribute ignorability rule (referred to by [P2552R3] as the “Second Ignorability Rule”). Many minor features are unable to take advantage of the syntax space that N2224 proposed to make available, and many minor annotations require (possibly contextual) keywords instead of attributes, which runs contrary to the guidance in N2761 that features that are “used in declarations or definitions only”, that are “of use to a limited audience only”, or that are “minor annotations” be standardized as attributes rather than keywords.

The use of (possibly contextual) keywords risks not only collisions with identifiers in pre-existing code, but also collisions with identifiers that users might eventually try to use. Such collisions can be resolved by a normative disambiguation rule, but the result might not be what the user expected, and the specification effort alone might be considerable. See Section 10 of [P2786R5] for an example of this phenomenon; I will discuss this example in more depth shortly.

I give some examples of how the Standard might benefit from the use of `@` followed by an *identifier* to introduce a non-ignorable annotation: that is, a syntactic entity similar to an attribute, but with mandatory semantics specified by the Standard. In effect, every occurrence of `@identifier` would be a keyword and all such keywords would be initially reserved for future standardization (that is, ill formed) until they are claimed by future proposals.

§ 3 Prior art in other programming languages

The `@` character is most closely associated with the Objective-C programming language. Several Objective-C keywords start with the `@` character, which clearly marks them as keywords. In addition, the fact that a keyword such as `@property` starts with the `@` character avoids the risk of changing the meaning of any C code that might use `property` as an ordinary identifier. The usage of the `@` character by Objective-C is therefore similar to what this paper suggests should be the future use of `@` by C++. (Objective-C also uses `@` as an operator for creating literals, but I am not aware of any interest in introducing a similar feature in C++.)

In Java, syntactic constructs that start with the `@` sign are called *annotations*. Java annotations are similar to C++ attributes in that the information they provide is usually not

essential to the meaning of a program, and some annotations have built-in semantics while others are given a meaning only by non-standard tools that consume source code or reflect on Java programs. However, unlike C++ attributes, Java annotations can have mandatory semantics, such as `@Override` (similar to the `override` contextual keyword in C++). Java’s use of the `@` character is therefore similar to what this paper suggests should be the future use of `@` by C++.

In Python, `@` introduces a *decorator*, which is a function that will be applied to the definition of a class or function, possibly changing its functionality. Python allows any function to be used as a decorator, and, therefore, there is no list of standard decorators; any built-in function could theoretically be used as a decorator. Some of the functions that are most well-known for their use as decorators include `@classmethod`, `@staticmethod`, `@property`, and `@functools.cache`. All of these examples arguably change the properties of the decorated entity in a major way, and their C++ analogues, if they existed, would therefore not satisfy the criteria in N2761 to be considered as minor annotations. Nevertheless, they do usually have the property that the decorated definition would make sense without the decorator, with a slightly different meaning. I believe that programmers who are familiar with Python decorators would find it natural if C++ used `@` to introduce non-ignorable annotations that can be applied to functions and classes. Eventually, C++ could introduce annotations that actually function similarly to Python decorators, using syntax like `@reflect(func)`, which would apply `func` to the reflection of the annotated entity.

The C# programming language uses `@` to introduce an identifier: for example, `@class` means an identifier spelled `class`, rather than the keyword `class`. Such usage would conflict with the objective of this paper, so I do not propose it.

§ 4 Example: contract annotations

According to [P2885R3], some users considered the attribute-like syntax proposed for Contracts to be too “heavy”, but others “consider the particular way in which the attribute-like syntax stands out visually to be a benefit, as it creates a clear separation between contract-checking annotations and other C++ code”. Note that the attribute-like syntax that was proposed was not actually an *attribute* syntax due to the presence of a colon after the identifier `pre`, `post`, or `assert`; therefore, it could not violate the attribute ignorability rule. However, the visual similarity of the attribute-like syntax to actual attributes could lead some readers to assume that a contract annotation is ignorable, and this was one of the reasons why some members of SG21 disliked the attribute-like syntax.

The “natural” syntax that ultimately gained consensus in SG21 did not suffer from the aforementioned issues with the attribute-like syntax, but also (in my opinion) does not provide the benefit of standing out visually and creating a clear separation in the way that the attribute-like syntax would have. The natural syntax also suffers from two disadvantages, discussion of which consumed a significant amount of SG21 time:

- It was not immediately obvious whether implementations would easily be able to parse the natural syntax, *i.e.*, determine when exactly the identifier pre or post has the meaning of introducing a contract annotation, rather than being an ordinary *identifier*.
- The natural syntax for an assertion cannot use the `assert` identifier, *i.e.*, the form `assert(conditional-expression)`, due to conflict with the `assert` macro in the Standard Library. Where assertion is present as a language feature in programming languages other than C++, it is overwhelmingly likely to be spelled simply `assert`, and it is unfortunate that C++ has to spell it `contract_assert` instead for backward compatibility.

Rostislav Khlebnikov suggested, but did not seriously propose, that contract annotations be introduced by the `@` keyword, *e.g.*, `@pre(x > 0)` or `@assert(x > 0)`. The latter would technically *still* be a breaking change, as I will discuss later, but one that is very unlikely to impact real code; it would therefore open up the possibility of using the preferred spelling `assert` rather than the more verbose `contract_assert`. In addition, this syntax for Contracts would, like the natural syntax, avoid the disadvantages of the attribute-like syntax, but would also have the benefit of standing out visually.

I do not actually propose to change the current consensus syntax for Contracts, but merely to provide an example of an option that could have been seriously considered if `@` were already considered by EWG to be suitable for introducing non-ignorable annotations.

§ 5 Example: trivial relocatability

[P2786R5] proposes a mechanism to explicitly specify whether a class type is trivially relocatable: `struct A trivially_relocatable {};` would define A to be a trivially relocatable class type, while `struct A trivially_relocatable(b) {};` would make A trivially relocatable if and only if `b` is `true` when considered as a contextually conveted constant expression. An issue with this contextual keyword is the type of “vexing parse” discussed in Section 10 of that paper:

```
struct A trivially_relocatable(bool(my_constexpr_value)) {
    // Is this a class definition, or the definition of a function named
    // `trivially_relocatable` with an elaborated type specifier for its
    // return
    // type and a `bool` parameter named `my_constexpr_value`?
};
```

The EWG decided that the new feature should not change the meaning of any code, even hypothetical code, that is currently well-defined, so that the above definition should be considered to define a function whenever it is a syntactically valid function definition. The CWG felt that requiring implementations to potentially consider the full content of the definition (*i.e.* everything between the braces) in order to determine whether the definition is of a class or a function would be unreasonably burdensome, and a considerable amount of effort was expended in determining how to word a more reasonable disambiguation rule

that would not require taking into account anything after the opening brace; this direction is pending EWG approval. Even assuming that the revised version of the disambiguation rule is approved by the EWG and the CWG, I consider it unfortunate that the choices are to either change the meaning of currently well-defined code (not approved by the EWG) or to disambiguate in the direction that is far less likely to have been intended by the user.

Consider, instead, if `trivially_relocatable` were to be preceded by the `@` character. The syntactic ambiguity would be entirely avoided, and the presence of `@` would alert the reader to the fact that `@trivially_relocatable` is a minor but non-ignorable annotation to the class definition. I do not actually propose in this paper to introduce the `@trivially_relocatable` syntax, but merely give it as an example of an option that the authors of P2786 might have considered if the EWG were known to be open to using `@` to introduce non-ignorable annotations.

§ 6 Example: profile annotations

[P2816R0] proposes an approach to improving the safety of the C++ language by defining a set of *profiles*, each of which enables a particular set of compile-time and/or run-time checks, and providing the ability to annotate C++ code to specify the profiles that would apply to it. The authors of P2816R0 have not yet published a revision that proposes a concrete syntax for profile annotations, but there has been some recent discussion about syntax on the SG23 reflector. Some Committee members have expressed opposition to the idea of using an attribute-based syntax for profile annotations because such annotations will be ignored by older compilers, and one member suggested the syntax `@enable(ranges)` as an example of a non-ignorable syntax for enabling the ranges profile.¹

§ 7 Example: `[[no_unique_address]]`

The `[[no_unique_address]]` attribute was intended to satisfy the attribute ignorability rule [P0840R0]. The Standard allows but does not require a member declared with `[[no_unique_address]]` to share its storage with another subobject; therefore, it might appear that the ignorability rule is satisfied. However, because `[[no_unique_address]]` normatively makes a non-static data member a potentially-overlapping subobject (§6.7.2 [intro.object]²p7.2), and the property of being potentially-overlapping triggers certain core language rules even if the implementation ignores the attribute for layout purposes, `[[no_unique_address]]` as initially specified was not ignorable. One reason for non-ignorability was resolved by [CWG2759], while another, described in [CWG2866], is still open, and there is no consensus within the CWG about how to resolve it. In addition, Microsoft will not implement `[[no_unique_address]]` with useful semantics until the next time a business decision can be taken to change their ABI in a non-backward-compatible fashion.

Due to the aforementioned problems with the `[[no_unique_address]]` attribute, some interest has been expressed on the EWG reflector in the idea of deprecating this attribute and replacing it with a keyword. Since this feature is minor enough to have been made an attribute in the first place, it may be a good candidate for a non-ignorable annotation, `@no_unique_address`. Such an annotation would still not have a mandatory effect on class layout, but would have mandatory effects on constructs that depend on whether a subobject is potentially-overlapping, and there would be no backward compatibility issue preventing any implementation from providing useful semantics for it.

§ 8 Making `@identifier` a preprocessing token

Since this paper doesn't propose any actual concrete annotations, all code that uses `@` outside a comment or literal would continue to be ill formed if this proposal were to be adopted. One might therefore wonder whether this paper proposes any normative changes to the Standard at all. The answer is yes, because in current C++, `@identifier` is two *preprocessing-tokens*, not one (§5.4 [\[lex.pptoken\]](#)). The implications of this fact are illustrated by the following example.

```
#define NDEBUG
#include <cassert>
#include <print>
#define STR(X) #X
#define STR2(X) STR(X)
#define A @assert(true)
#define M STR2(A)
int main() {
    std::print("{} ", M);
}
```

In current C++, the above program prints `@((void)0)`: prefixing `assert` with `@` does not prevent expansion of `assert`. If `@assert` were a single preprocessing token, then it would not be the name of a function-like macro, so the above program would print `@assert(true)`.

The benefit of making `@identifier` a single token is that whenever a concrete annotation is eventually introduced into the language, it could be used in any C++ program even if the *identifier* is already defined as a macro. Although SG21 has already decided that contract assertions should be introduced via a new keyword, `contract_assert`, it is possible that there will be enthusiasm for the additional syntax `@assert` at a later time.

I do not propose to make a lone `@` ill formed as a preprocessing token. Under this proposal, `@` that is not immediately followed by an identifier would remain a preprocessing token (see the grammar in §5.4 [\[lex.pptoken\]](#)) and would therefore be eligible to be concatenated with an identifier using the `##` operator.

This change to the C++ preprocessor may cause a one-time breakage now, *i.e.*, for programs that actually rely on something like the behavior above, where `@((void)0)` is printed. I consider it highly unlikely that any programs that are not compiler test suites are actually relying on this. However, the expected number of such programs in existence is more likely to increase than decrease over time, so if the Committee wants to maximize the ability to leave `@identifier` available for future keywords denoting non-ignorable annotations, I believe that `@identifier` should be changed to be a single token now.

Note that making `@identifier` a single token will introduce a difference between the C++ preprocessor and the C preprocessor. While the difference is unfortunate, we have precedent in the form of user-defined literals; see §C.6.2 [\[diff.cpp03.lex\]p2](#).³

§ 9 Scoped annotation keys

An open question is whether scoped annotation keys should be supported, such as `@gnu::foo`, for introducing vendor-specific annotations. Such vendor-specific annotations would necessarily differ from hypothetical standard annotations, since a standard annotation that is not recognized by the implementation would be ill formed, whereas a vendor-specific annotation that is not recognized by the implementation should be ignored, since making it ill formed would fragment the C++ language into vendor-specific dialects. Thus, it may be undesirable to use the same syntax for both ignorable and non-ignorable annotations (depending on whether the name following the `@` is qualified). Introducing ignorable, vendor-specific annotations also makes the feature harder to explain: `@` could no longer be described as a character that is simply present in the spelling of certain keywords (as in Objective-C).

If we introduce scoped annotation keys, then we must take care to prevent a certain lexical ambiguity: for example, if `@foo` is a valid annotation key whose syntax does not have any kind of argument clause, then `@foo::bar x;` could mean that `@foo` is an annotation of the declaration `::bar x;`, or that `@foo::bar` is an annotation of the expression statement `x;`. To resolve this ambiguity, we could simply say that `@` followed by a qualified name is also a single preprocessing token; the “maximal munch” rule (§5.4 [\[lex.pptoken\]p3.3](#)) then implies that `@foo::bar x;` is always interpreted as `@foo::bar` annotating `x;`. This choice implies that whitespace must be used in order to obtain the other interpretation, *i.e.*, `@foo ::bar x;`.

Since vendor-specific attributes are not subject to the ignorability rule, implementations already have full freedom to give such attributes any semantics they consider desirable. For example, the syntax `@gnu::foo` does not need to be supported because GCC can already define `[[gnu::foo]]` to mean whatever they want.

However, the one reason why we might want to consider supporting vendor-specific annotations (which must, as explained above, be ignorable) is for purposes of reflection. For example, consider a reflection-based static analysis library, `foo`, which might wish to make use of the annotation `@foo::bar` and benefit from a guarantee that this annotation

will be visible to reflection. It might not be possible to use attributes for this purpose, since Clang discards unrecognized attributes during parsing, and they do not appear in the AST.

Considering the possible disadvantages described above, and the fact that some additional design questions remain for how to specify ignorable scoped annotations (for example, what kinds of argument clauses they could take), I do not take a position in this proposal as to whether ignorable scoped annotations should be allowed at all. A future proposal to introduce ignorable scoped annotations should consider design alternatives, such as an unscoped annotation key that can be used to introduce arbitrary information into the AST (e.g., `@annotate(foo::bar, "value_for_bar")`). Instead, I propose to leave this design space open by specifying that anything that looks like a scoped annotation key is ill formed.

§ 10 Wording

Edit §5.4 [[lex.pptoken](#)]:

```
preprocessing-token:
    header-name
    import-keyword
    module-keyword
    export-keyword
    identifier
    annotation-key
    pp-number
    character-literal
    user-defined-character-literal
    string-literal
    user-defined-string-literal
    preprocessing-op-or-punc
    each non-whitespace character that cannot be one of the above
```

Each preprocessing token that is converted to a token shall have the lexical form of a keyword, an annotation key, an identifier, a literal, or an operator or punctuator.

[...] The categories of preprocessing tokens are: header names, placeholder tokens produced by preprocessing `import` and `module` directives (*import-keyword*, *module-keyword*, and *export-keyword*), identifiers, annotation keys, preprocessing numbers, character literals (including user-defined character-literals), string literals (including user-defined string literals), preprocessing operators and punctuators, and single non-whitespace characters that do not lexically match the other preprocessing token categories. [...]

[...]

Edit §5.6 [lex.token]:

annotation-key:
@ identifier
annotation-key :: identifier

token:
identifier
keyword
annotation-key
literal
operator-or-punctuator

There are ~~five~~six kinds of tokens: identifiers, keywords, annotation keys, literals, operators, and other separators. [...]

[Note 1: [...] — end note]

[Note 2: All *annotation-keys* appearing outside an *attribute* ([dcl.attr.grammar]) are reserved for future standardization. — end note]

In §15.11 [cpp.predefined], add a feature test macro named `__cpp_annotations`.

§ 11 Policy for use of *annotation-keys*

If EWG reaches consensus to forward the wording changes proposed by this paper to CWG, I propose that an additional poll be taken on whether EWG encourages work in the direction of proposing a policy for EWG to adopt with respect to criteria for determining whether an *annotation-key* (as opposed to a regular keyword) is appropriate syntax for a new non-ignorable feature. I do not plan to formally propose any such policy myself, but I would like to suggest that some of the criteria listed in N2761 might be a good starting point.

§ 12 Acknowledgements

Rostislav Khlebnikov suggested this usage for the `@` character. Lauri Vasama pointed out the potential lexical ambiguity arising from scoped annotation keys.

§ 13 References

[CWG2759] Richard Smith. 2020-11-10. `[[no_unique_address]]` and common initial sequence.

<https://wg21.link/cwg2759>

[CWG2866] Brian Bi. 2023-11-12. Observing the effects of `[[no_unique_address]]`.

<https://wg21.link/cwg2866>

[N2224] Alisdair Meredith. 2007-03-12. Seeking a Syntax for Attributes in C++09.

<https://wg21.link/n2224>

[N2761] J. Maurer, M. Wong. 2008-09-18. Towards support for attributes in C++ (Revision 6).

<https://wg21.link/n2761>

[P0840R0] Richard Smith. 2017-10-16. Language support for empty objects.

<https://wg21.link/p0840r0>

[P2552R3] Timur Doumler. 2023-06-14. On the ignorability of standard attributes.

<https://wg21.link/p2552r3>

[P2558R2] Steve Downey. 2023-02-08. Add @, \$, and ` to the basic character set.

<https://wg21.link/p2558r2>

[P2786R5] Mungo Gill, Alisdair Meredith. 2024-04-09. Trivial Relocatability For C++26.

<https://wg21.link/p2786r5>

[P2816R0] Bjarne Stroustrup, Gabriel Dos Reis. 2023-02-16. Safety Profiles: Type-and-resource Safe programming in ISO Standard C++.

<https://wg21.link/p2816r0>

[P2885R3] Timur Doumler, Joshua Berne, Gašper Ažman, Andrzej Krzemiński, Ville Voutilainen, Tom Honermann. 2023-10-05. Requirements for a Contracts syntax.

<https://wg21.link/p2885r3>

1. On the other hand, some members consider it desirable for profile annotations to be ignored by older compilers. I would not expect a syntax that uses the @ character to appeal to those members. ↩
2. All citations to the Standard are to working draft N4981 unless otherwise specified. ↩
3. While the indicated paragraph of the Standard discusses a difference between the preprocessor of current C++ and that of C++03, the same difference exists between C++ and C, because C does not have user-defined literals. ↩